Звіт з лабораторної роботи 2

Побудова діаграми класів предметної області

Виконав студент групи ІПЗ-23-3
Некрасов Артур Дмитрович

2026

1 Мета та вихідні вимоги

Мета роботи — побудувати діаграму класів університетської бібліотеки на основі Use Case моделі лабораторної роботи №1. У результаті визначено 11 класів, зокрема один абстрактний, і один інтерфейс. Кожен клас має щонайменше 3 власні атрибути та 2 операції; застосовано всі п’ять потрібних типів відношень [1, 2].

Зв’язок із попередньою роботою

Предметна область та правила збережені: читач шукає видання, переглядає опис, бронює примірник, завантажує дозволений файл, продовжує позичку або скасовує резерв. Бібліотекар веде каталог, оформлює видачу й повернення; адміністратор керує обліковими записами. Зовнішній сервіс приймає підтвердження бронювання.

Діаграма класів деталізує об’єкти, стан і поведінку, необхідні для реалізації вимог, визначених у першій роботі.

Правила, які враховано в моделі

1. Активне бронювання діє 48 годин; читач має не більше 3 активних бронювань.
2. Одночасно дозволено не більше 5 відкритих позичок. Видача оформлюється на 14 календарних днів.
3. Непрострочену позичку дозволено продовжити один раз на 14 днів від поточного строку повернення.
4. Право читача залежить від чинності квитка, блокування та відсутності прострочених позичок.
5. Для завантаження потрібні авторизований читач, наявний файл і дозвіл на доступ.
6. Недоступність зовнішнього сервісу не скасовує бронювання: повідомлення зберігається для повторної спроби.

Видимість і типи даних

Атрибути сутностей позначено як private «−», спільні атрибути абстрактної Особи — protected «#», операції — public «+». Символ «~» означає пакетну видимість; у цій моделі він не потрібний. UUID є ідентифікатором, Date — датою, DateTime — моментом часу, URI — адресою файлу, Bytes — його вмістом. Set<T> і List<T> задають колекції, а [0..1] біля атрибута — необов’язкове значення [1, 3].

Назви класів на діаграмі подано українською, а короткі англійські імена атрибутів, операцій і псевдоніми класів використано для однозначного запису сигнатур.

2 Перелік класів та інтерфейсу

У підрахунку атрибутів і методів враховано власні оголошення кожного класу, без успадкованих членів. Інтерфейс задає контракт із двох операцій і не містить штучно доданих полів стану.

Таблиця 1 — Склад моделі та повнота опису

Клас і псевдонім	Тип та призначення	Атрибути / методи
Особа Person	абстрактний клас Спільні дані та контактні операції читача і працівників.	4 / 2
Читач Reader	клас Перевірка права читача, бронювання та доступ до електронних матеріалів.	4 / 3
Бібліотекар Librarian	клас Реєстрація видачі та повернення конкретного примірника.	3 / 2
Адміністратор Administrator	клас Оновлення даних, ролей і блокування користувачів.	3 / 3
ЧитацькийКвиток LibraryCard	клас Особистий квиток читача з терміном чинності та статусом.	3 / 2
Каталог Catalogue	клас Організація пошуку та підтримання бібліографічних записів.	3 / 3
Видання Edition	клас Опис видання та дані про дозволений електронний файл.	6 / 2
Примірник Copy	клас Фізична одиниця фонду з унікальним інвентарним номером і станом.	4 / 2
Бронювання Reservation	клас Резерв конкретного примірника за читачем на 48 годин.	4 / 3
Позичка Loan	клас Облік видачі, строку повернення та одноразового продовження.	5 / 3
Сповіщення Notification	клас Збереження підтвердження бронювання, стану й повторних спроб надсилання.	6 / 2

Клас і псевдонім	Тип та призначення	Атрибути / методи
Продовжуваний Renewable	інтерфейс Контракт перевірки можливості та виконання продовження.	— / 2

Особа не має безпосередніх екземплярів. Читач, Бібліотекар і Адміністратор успадковують її ідентифікатор, ПІБ, email, набір ролей та контактні операції. Позичка реалізує контракт Продовжуваний.

3 Загальний огляд діаграми класів

Діаграму відкрито й перевірено в draw.io. Для огляду зв’язків атрибути та методи приховано лише в цьому поданні. Далі наведено збільшені подання тих самих класів з усіма членами. Усі п’ять ілюстрацій у звіті — справжні знімки вікна редактора.

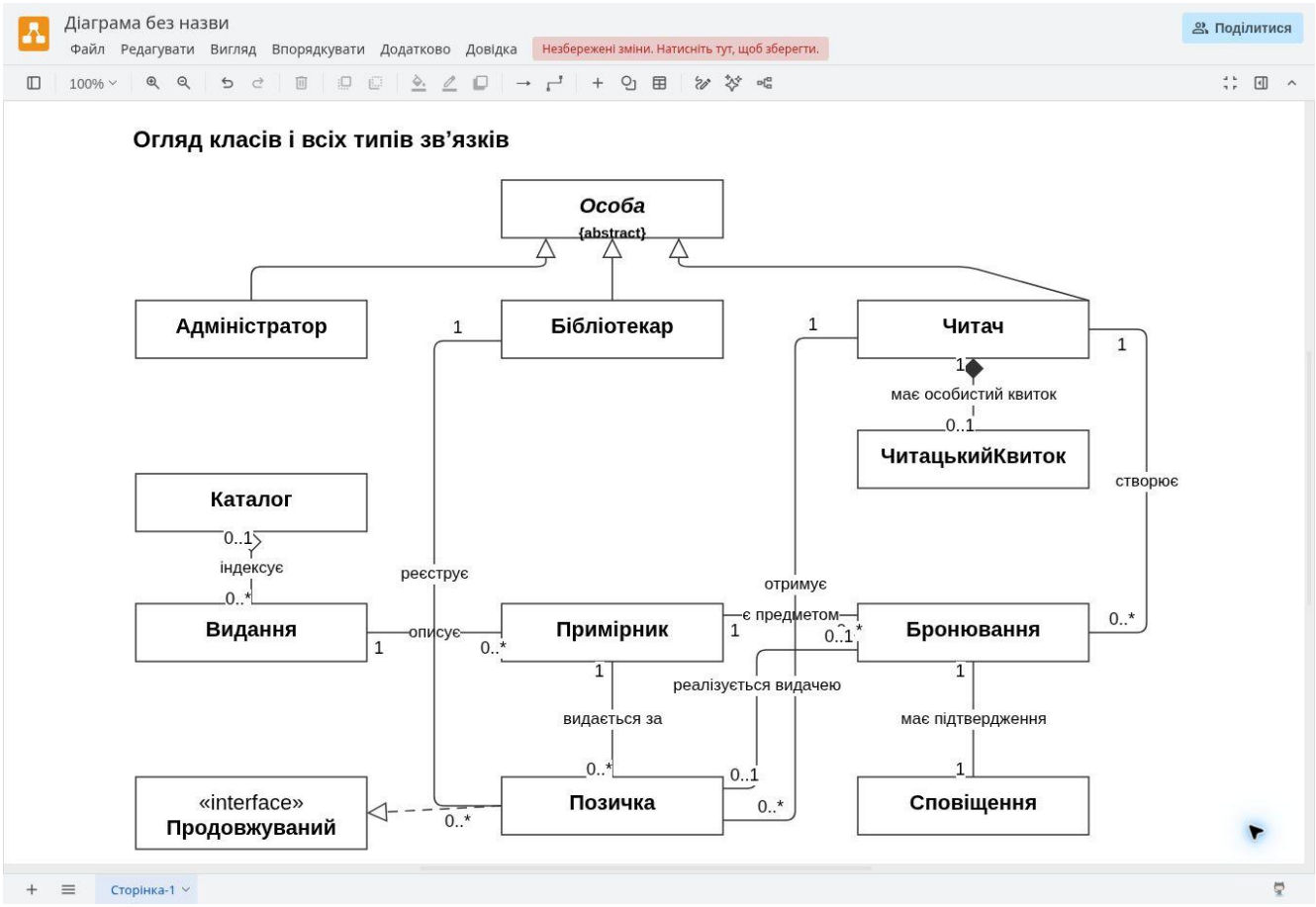


Рисунок 1 — Скріншот загального огляду в draw.io

Читання основних зв’язків

Суцільні лінії без спеціального символу позначають асоціації. Порожній ромб розміщено біля Каталогу, зафарбований — біля Читача. Три суцільні лінії з порожнім трикутником спрямовано від нащадків до Особи. Пунктир із порожнім трикутником спрямовано від Позички до інтерфейсу Продовжуваний [1, 3, 4].

Числа біля кінців ліній визначають множинність. Наприклад, зв’язок Reader «1» — «0..*» Loan означає, що кожна позичка стосується одного читача, а читач може мати довільну кількість позичок за весь час. Окремі обмеження 3 і 5 застосовуються до активних записів, а не до історії.

4 Особи та їхні ролі

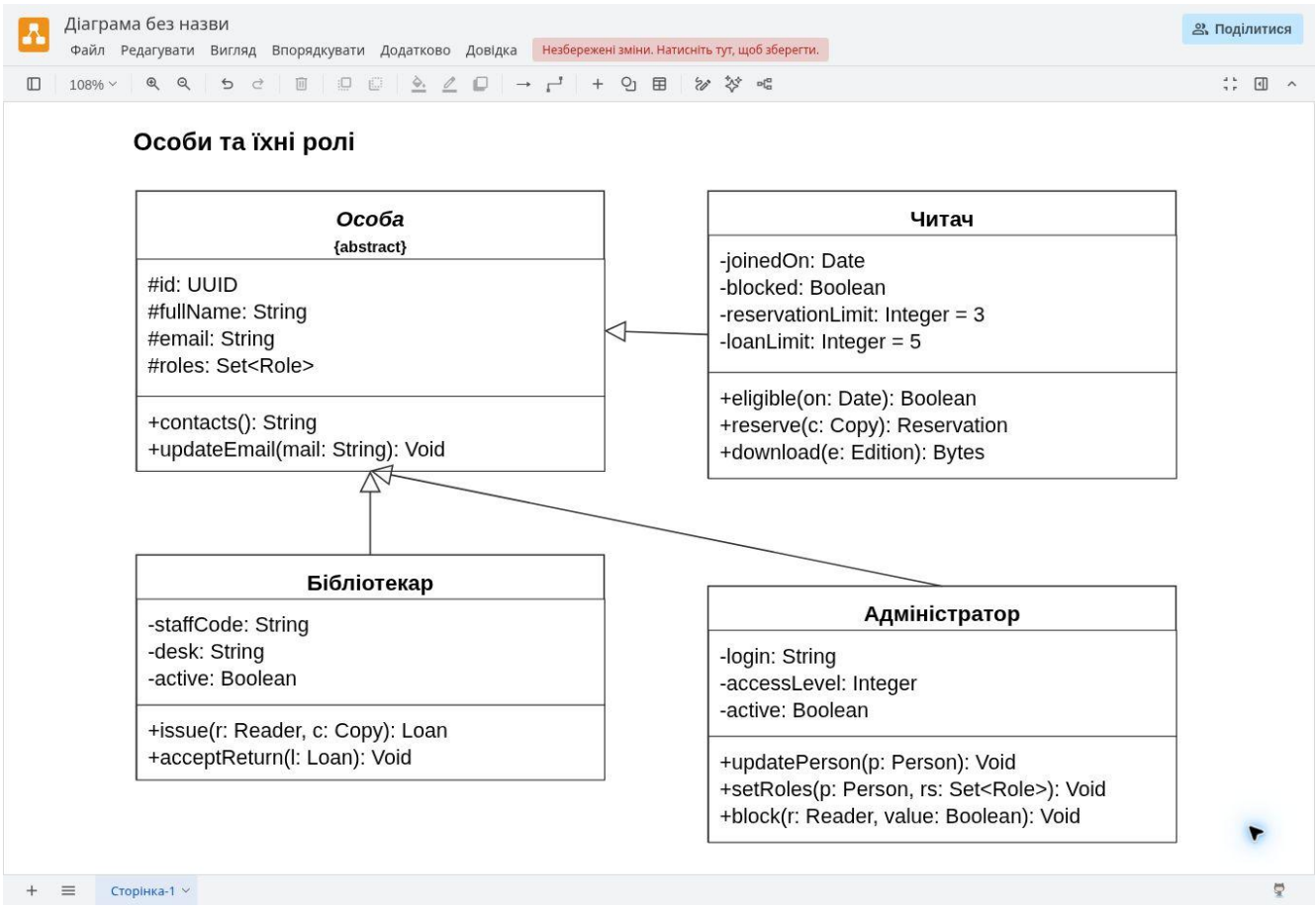


Рисунок 2 — Скріншот узагальнення класів осіб

Спільні дані та спеціалізація

Особа зберігає id, fullName, email і roles. Назву абстрактного класу виділено курсивом і додатково позначено {abstract}. Захищена видимість спільних атрибутів дає змогу використовувати їх у нащадках, а зовнішню взаємодію задають публічні операції.

Читач має дату реєстрації, ознаку блокування та ліміти активних операцій. eligible() об’єднує перевірку квитка, блокування й прострочень; reserve() ініціює створення бронювання. Завантаження проходить через download() після перевірки права доступу до файлу.

Бібліотекар містить службовий код, пункт обслуговування та ознаку активності. issue() оформлює позичку, acceptReturn() реєструє повернення. Адміністратор оновлює персональні дані, змінює ролі й блокування. Звичайний користувач каталогу не потребує окремого класу: пошук і перегляд не зберігають персонального стану.

5 Каталог та фізичний фонд

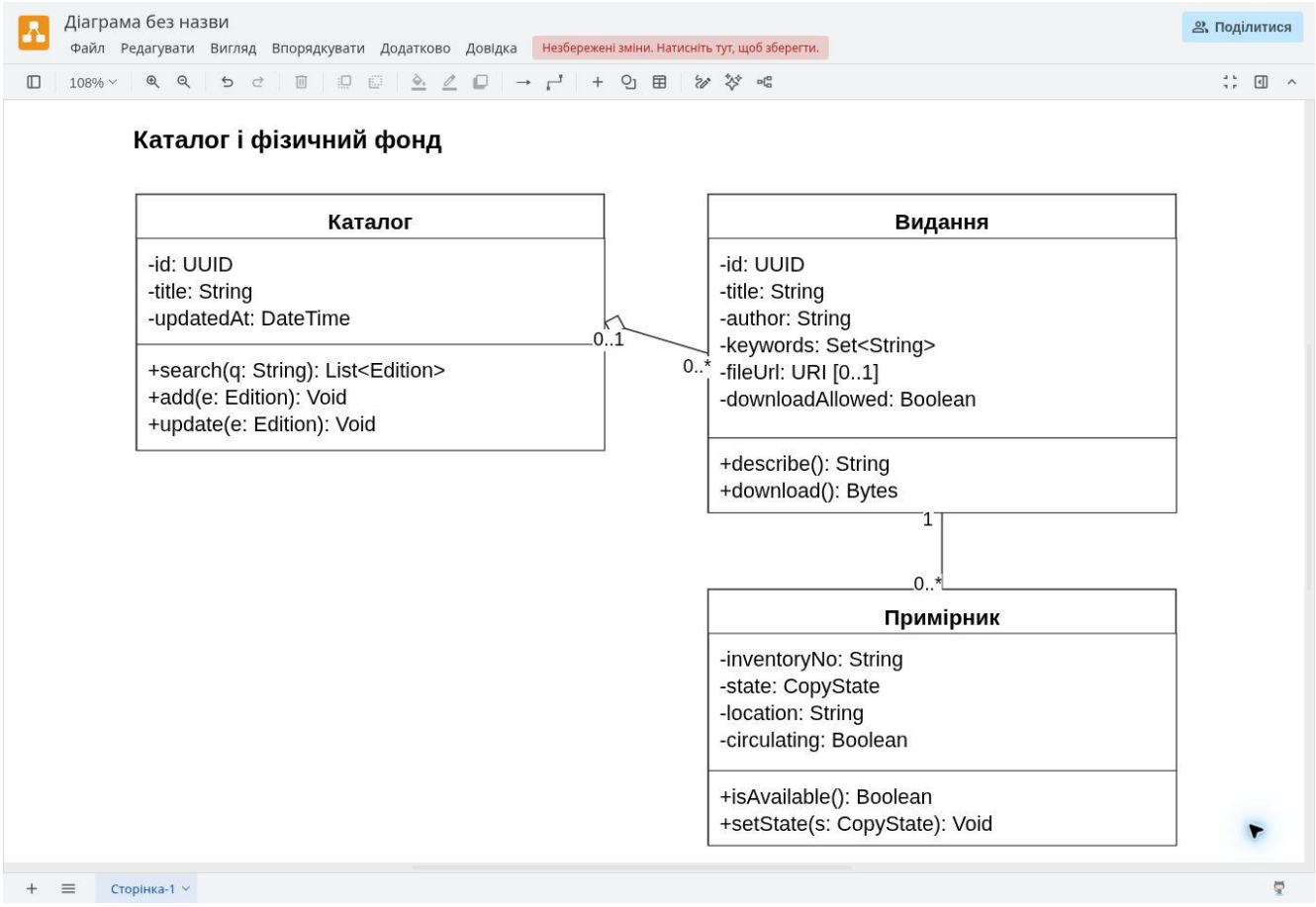


Рисунок 3 — Скріншот каталогу та бібліотечного фонду

Опис видання та конкретна книга

Каталог групує бібліографічні записи й надає search(), add() та update(). Видання містить назву, автора й ключові слова. fileUrl має множинність [0..1]: електронний файл може бути відсутнім. downloadAllowed визначає, чи дозволений доступ до наявного файлу.

Примірник — окрема фізична книга. inventoryNo є унікальним інвентарним номером, state відображає доступність, location задає місце отримання, а circulating — можливість видачі. Одне видання може мати 0..* примірників; кожен примірник пов’язаний з одним виданням.

Агрегація Каталог — Видання є слабким групуванням. Примірник пов’язаний із Виданням звичайною асоціацією: вилучення опису з активного каталогу не повинно автоматично знищувати облік фізичних книг та їх історію.

6 Бронювання та підтвердження

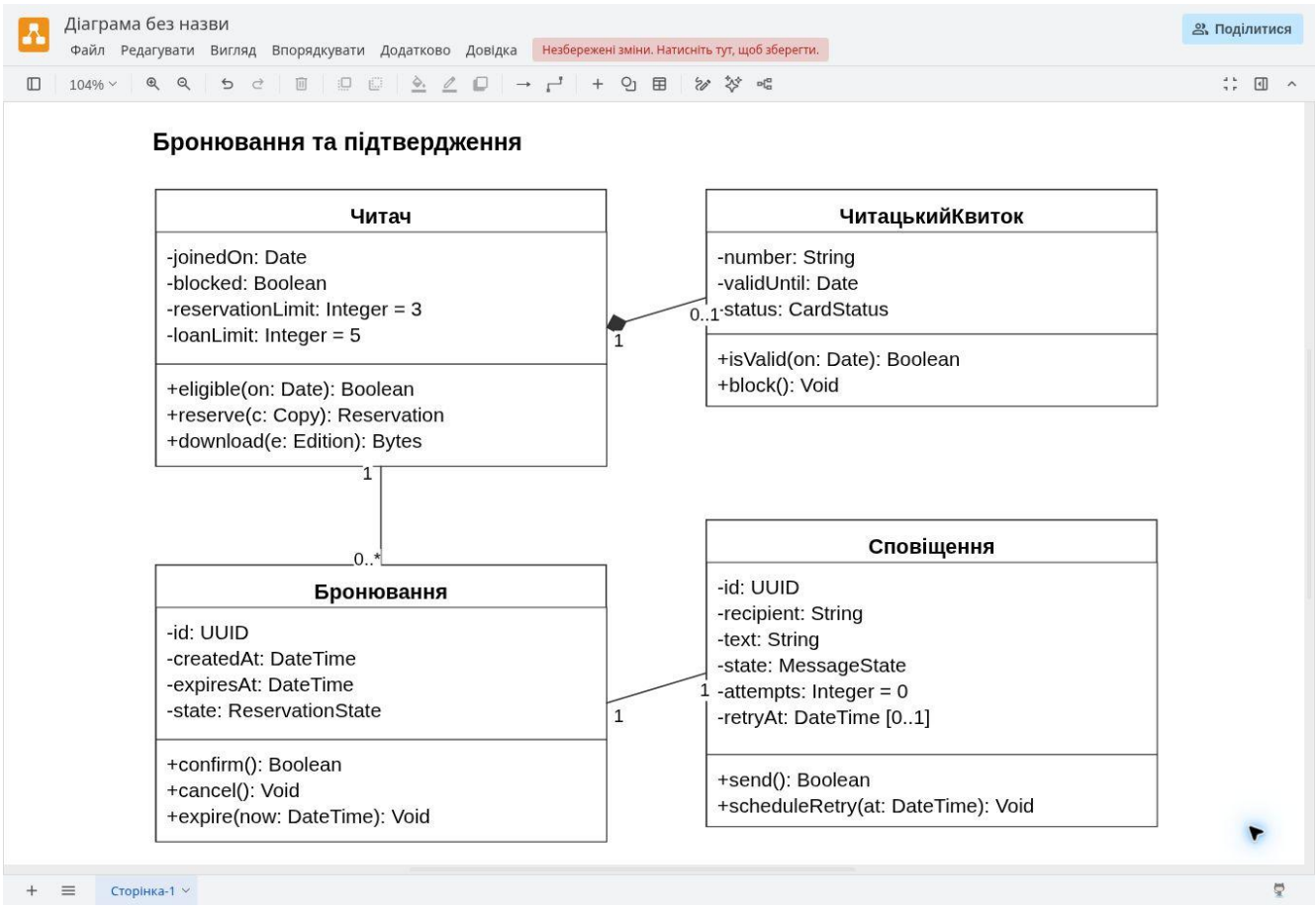


Рисунок 4 — Скріншот бронювання та композиції квитка

Особистий квиток і резерв

ЧитацькийКвиток має власний номер, дату чинності та статус. Він належить одному Читачеві й не передається іншому власникові. У читача може ще не бути активованого квитка, тому з боку квитка задано 0..1; такому читачеві eligible() відмовляє в обслуговуванні.

Бронювання зберігає createdAt і expiresAt; для підтвердженого резерву expiresAt = createdAt + 48 годин. confirm() фіксує резерв, cancel() скасовує його, expire() завершує прострочений резерв. Історія бронювань зберігається, тому зв’язок із читачем має 0..*.

Підтвердженому бронюванню відповідає одне Сповіщення. Адресат і текст фіксуються в повідомленні. send() виконує спробу передавання сервісу, а scheduleRetry() визначає повторну спробу. attempts і retryAt дають змогу врахувати збій без скасування резерву. SENT означає передавання сервісу, а не факт прочитання читачем.

7 Позичка та контракт продовження

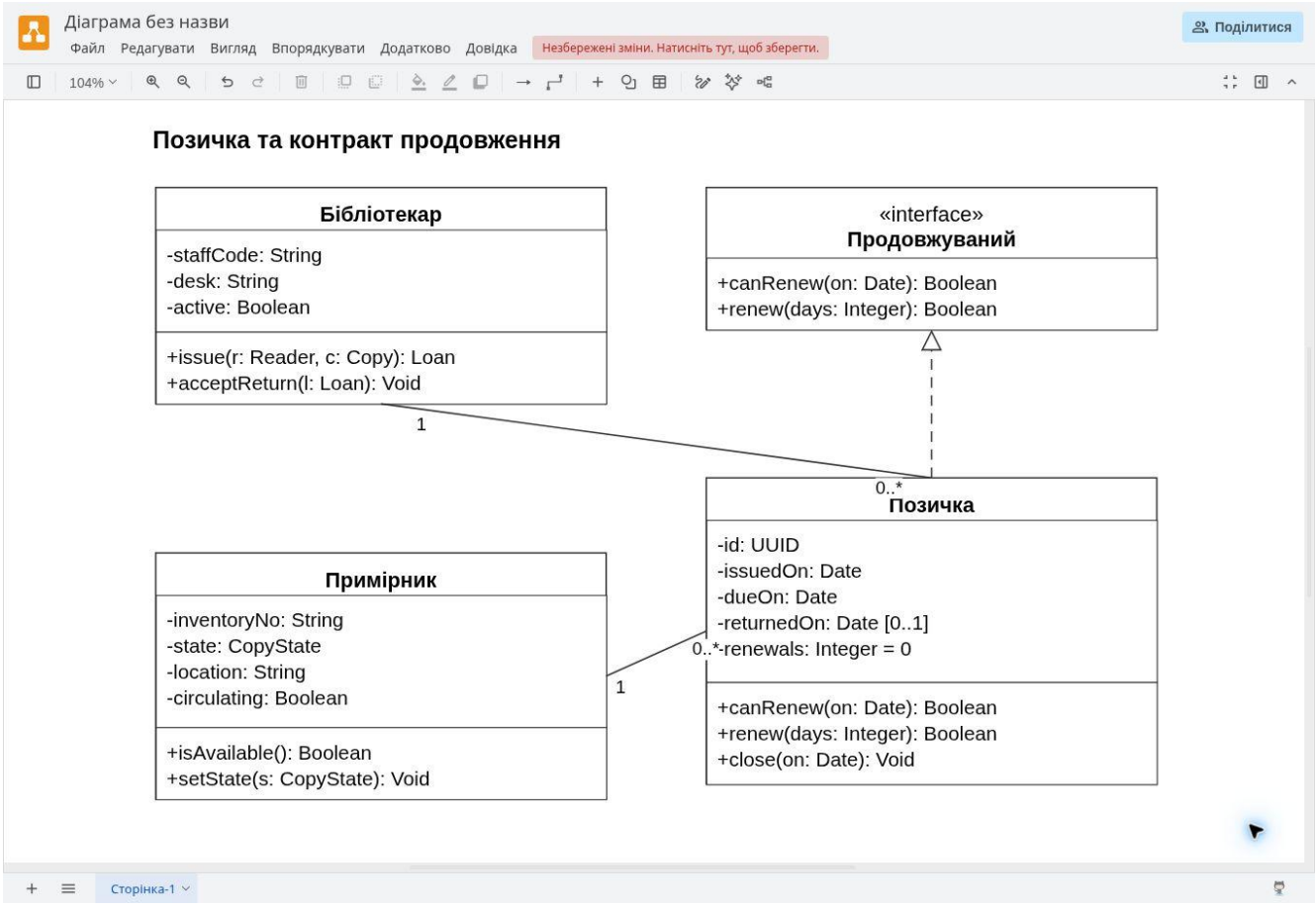


Рисунок 5 — Скріншот реалізації інтерфейсу Продовжуваний

Видача повернення та продовження

Позичка зберігає дати видачі та повернення, а також лічильник продовжень. returnedOn має множинність [0..1]: поки книгу не повернуто, значення відсутнє. close() установлює фактичну дату повернення. Одна фізична книга може мати багато позичок у різні моменти, але не більше однієї відкритої одночасно.

Інтерфейс Продовжуваний задає canRenew(on) та renew(days), а клас Позичка оголошує обидві операції з тими самими сигнатурами. Реалізацію позначено пунктирною лінією з порожнім трикутником біля інтерфейсу.

Для бібліотеки renew() приймає лише 14 днів і виконується один раз: renewals змінюється з 0 на 1, dueOn збільшується від попереднього строку. До зміни перевіряються право читача, відсутність прострочення та circulating примірника. Нової позички під час продовження не створюють.

8 Відношення та множинність

У таблиці A і B означають класи в порядку запису. Множинності наведено біля A та біля B відповідно. Для узагальнення й реалізації вони не застосовуються.

Таблиця 2 — Усі відношення моделі

Тип	Класи A та B	Біля A / B	Зміст
Узагальнення	Reader, Librarian, Administrator → Person	—	Три спеціалізації спільного абстрактного класу.
Реалізація	Loan → Renewable	—	Позичка виконує контракт продовження.
Композиція	Reader — LibraryCard	1 / 0..1	Один власник; необов’язковий особистий квиток.
Агрегація	Catalogue — Edition	0..1 / 0..*	Каталог групує незалежні бібліографічні записи.
Асоціація	Edition — Copy	1 / 0..*	Одне видання описує багато фізичних примірників.
Асоціація	Reader — Reservation	1 / 0..*	Бронювання належить одному читачеві.
Асоціація	Copy — Reservation	1 / 0..*	Кожен резерв стосується одного примірника; врахована історія.
Асоціація	Reader — Loan	1 / 0..*	Читач має історію власних позичок.
Асоціація	Copy — Loan	1 / 0..*	Примірник багаторазово видається в різні моменти.
Асоціація	Librarian — Loan	1 / 0..*	Кожну видачу реєструє один бібліотекар.
Асоціація	Reservation — Loan	0..1 / 0..1	Резерв може завершитися видачею; видача можлива й без резерву.
Асоціація	Reservation — Notification	1 / 1	Для підтвердженого резерву збережено одне повідомлення.

Обмеження активних записів

Для одного примірника кількість активних бронювань не перевищує 1, а відкритих позичок — 1. Активне бронювання та відкрита позичка не можуть одночасно належати тому самому примірнику: під час видачі резерв погашається. Ліміти читача перевіряються як count(activeReservations) ≤ 3 і count(openLoans) ≤ 5.

Обидва кінці Reservation — Loan мають 0..1: резерв може бути скасований чи сплинути без видачі; позичку можна оформити без попереднього бронювання. Цей зв’язок не задає спільного життєвого циклу.

9 Обґрунтування відношень ціле та частина

Для кожного ромба застосовано тест із завдання: чи має сенс частині продовжувати існувати незалежно після видалення цілого [1].

Таблиця 3 — Обґрунтування агрегації та композиції

Відношення	Вибір	Перевірка життєвого циклу
Catalogue — Edition	Агрегація	Так, бібліографічний запис зберігає зміст незалежно від конкретного каталогу. Його можна тимчасово не індексувати або включити до іншого каталогу. Видалення групування не повинно видаляти саме видання. Порожній ромб розміщено біля Catalogue; 0..1 дозволяє запис без активного каталогу.
Reader — LibraryCard	Композиція	Ні, особистий квиток не має самостійного власника поза конкретним профілем читача і не може одночасно належати кільком читачам. Якщо профіль остаточно припиняє існувати в цій моделі, його квиток також припиняє існувати. Зафарбований ромб розміщено біля Reader. Тимчасове блокування чи архівування профілю не є знищенням об’єкта.

Чому інші зв’язки залишено асоціаціями

Позичка не є структурною частиною читача або книги: це самостійний запис про подію видачі. Його потрібно зберігати й після закриття. Аналогічно, бронювання та повідомлення мають власні ідентифікатори й стани; скасування резерву не видаляє підтвердження з історії.

Видання — Примірник не позначено композицією. Книга є фізичною одиницею фонду; зміна чи архівування бібліографічного опису не знищує книгу. За наявності примірників та історії запис видання архівують або переприв’язують у контрольованому процесі, зберігаючи цілісність посилань.

Життєвий цикл та збереження історії

Скасування бронювання змінює Reservation.state на CANCELLED; повернення встановлює Loan.returnedOn. Це зміни стану наявних об’єктів, а не їх фізичне видалення. Позначення композиції саме по собі не є командою видаляти з бази історичні позички або пов’язані документи.

10 Узгодженість із варіантами використання

Кожному з 12 Use Case лабораторної роботи №1 відповідають класи, операції та дані нової моделі. Англійські імена нижче — псевдоніми з таблиці 1 і повного .puml.

Таблиця 4 — Трасованість функціональних вимог

UC	Мета	Класи та відповідальна поведінка
UC01	Шукати видання	Catalogue.search(q); Edition.title, author, keywords.
UC02	Переглянути опис	Edition.describe(); Copy.state і location; Edition.fileUrl та downloadAllowed.
UC03	Забронювати примірник	Reader.reserve(); Reservation.confirm(); перевірка Copy.isAvailable() і Reader.eligible(); створення Notification.
UC04	Перевірити право читача	Reader.eligible(); LibraryCard.isValid(); Reader.blocked; перевірка прострочених Loan за dueOn та returnedOn.
UC05	Завантажити електронну версію	Reader.download(e) і Edition.download(); обов’язкові fileUrl та downloadAllowed = true.
UC06	Продовжити термін позички	Loan.canRenew() та renew(14) через контракт Renewable; Reader.eligible(), Copy.circulating.
UC07	Скасувати бронювання	Reservation.cancel(); перевірка власника через Reader — Reservation; Copy.setState(AVAILABLE).
UC08	Оформити видачу примірника	Librarian.issue(); створення Loan, строк dueOn, погашення резерву й Copy.setState(LOANED).
UC09	Зареєструвати повернення	Librarian.acceptReturn(); Loan.close(); Copy.setState() за результатом огляду.
UC10	Керувати каталогом	Catalogue.add() і update(); дані Edition та Copy; право бібліотекаря задається Role.LIBRARIAN.
UC11	Керувати обліковими записами	Administrator.updatePerson(), setRoles() і block(); Person.roles та Reader.blocked.
UC12	Сповістити читача	Notification.send(), scheduleRetry(); recipient, text, state, attempts і retryAt.

Відношення include та extend залишаються у поведінковій моделі Use Case. На діаграмі класів вони не дублюються: тут показано структуру об’єктів, яка забезпечує відповідні сценарії.

11 Перевірка даних і ключових понять

Таблиця 5 — Ключові іменники зі специфікацій UC03 UC06 UC08

Поняття	Представлення в моделі
Читач, ПІБ, контакт	Reader; успадковані Person.fullName та email.
Квиток, чинність, блокування	LibraryCard.number, validUntil, status; Reader.blocked.
Видання, електронний файл	Edition.title, author, fileUrl, downloadAllowed.
Примірник, номер, місце видачі	Copy.inventoryNo, state, location; Librarian.desk.
Бронювання, підтвердження, строк	Reservation.id, state, createdAt, expiresAt; зв’язок із Notification.
Позичка, повернення, продовження	Loan.id, issuedOn, dueOn, returnedOn, renewals; контракт Renewable.
Повідомлення, адресат, повторна спроба	Notification.recipient, text, attempts, retryAt і send(); зовнішній сервіс є межею інтеграції.

Таблиця 6 — Допустимі значення типів переліку

Тип	Значення
Role	READER, LIBRARIAN, ADMINISTRATOR
CardStatus	ACTIVE, BLOCKED, EXPIRED
CopyState	AVAILABLE, RESERVED, LOANED, WITHDRAWN
ReservationState	ACTIVE, CANCELLED, EXPIRED, FULFILLED
MessageState	PENDING, SENT, RETRY

Типи переліку повністю оголошено в .puml; для читабельності їх блоки приховано в графічному поданні. Це типи значень, а не додаткові сутності, до яких застосовується вимога про 3 атрибути та 2 методи.

Результат перевірки повноти

Підрахунок власних членів підтвердив вимогу для всіх 11 класів: мінімум 3 атрибути і 2 операції. Визначено 14 відношень: 8 асоціацій, 1 агрегацію, 1 композицію, 3 узагальнення та 1 реалізацію. Сигнатури двох операцій Renewable збігаються з операціями Loan. Повний код PlantUML успішно згенерував діаграму; візуально перевірено п’ять подань у редакторі.

Висновки

Побудовано структурну модель бібліотеки з 11 класів та одного інтерфейсу. Усі класи мають потрібний набір типізованих атрибутів і операцій. Обґрунтовано кожне відношення ціле — частина, задано множинність з урахуванням історії та окремих лімітів активних записів. Модель

узгоджена з усіма 12 Use Case першої роботи; її подання зафіксовано справжніми скріншотами draw.io.

Використані джерела

[1] Лабораторна_робота_02.html та Лаб2_діаграма_класів.puml. Надані завдання й еталонний приклад.

[2] Некрасов А. Д. Звіт із лабораторної роботи №1. Система університетської бібліотеки. ІПЗ-23-3, 2026.

[3] PlantUML. Class diagram. https://plantuml.com/class-diagram

[4] draw.io. UML class diagrams. https://www.drawio.com/docs/diagram-types/uml/class-diagrams/